

```

import math
import os
import sys

def create_board():
    # Pre-populating the board based on your simulation instructions:
    # 1: O, 2: empty, 3: X, 4: X, 5: empty, 6: empty, 7: X, 8: O, 9: O
    return ['O', ' ', 'X', 'X', ' ', ' ', 'X', 'O', 'O']

def print_board(board):
    os.system('cls' if os.name == 'nt' else 'clear')
    print("\n")
    print(f" {board[0]} | {board[1]} | {board[2]} ")
    print("---|---|---")
    print(f" {board[3]} | {board[4]} | {board[5]} ")
    print("---|---|---")
    print(f" {board[6]} | {board[7]} | {board[8]} ")
    print("\n")

def is_moves_left(board):
    return ' ' in board

def evaluate(board):
    win_states = [[0, 1, 2], [3, 4, 5], [6, 7, 8], # Rows
                  [0, 3, 6], [1, 4, 7], [2, 5, 8], # Columns
                  [0, 4, 8], [2, 4, 6]             # Diagonals
    ]
    for condition in win_states:
        if board[condition[0]] == board[condition[1]] == board[condition[2]]:
            if board[condition[0]] == 'O':
                return 10
            elif board[condition[0]] == 'X':
                return -10
    return 0

def minimax(board, depth, is_max):
    score = evaluate(board)

    if score == 10:

```

```

        return score - depth
    if score == -10:
        return score + depth
    if not is_moves_left(board):
        return 0

    if is_max:
        best = -math.inf
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                best = max(best, minimax(board, depth + 1, False))
                board[i] = ' '
        return best
    else:
        best = math.inf
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                best = min(best, minimax(board, depth + 1, True))
                board[i] = ' '
        return best

def find_best_move(board):
    best_val = -math.inf
    best_move = -1
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            move_val = minimax(board, 0, False)
            board[i] = ' '
            if move_val > best_val:
                best_move = i
                best_val = move_val
    return best_move

def play_game():
    board = create_board()
    print("Welcome to the Tic-Tac-Toe Simulation!")
    print("Positions are numbered 1-9 from top-left to bottom-right.")

```

```

print("Initial setup complete. It is X's turn.")
input("\nPress Enter to make your move...")

while True:
    print_board(board)

    score = evaluate(board)
    if score == -10:
        print("Congratulations! You won!")
        break
    if score == 10:
        print("The AI wins! Better luck next time.")
        break
    if not is_moves_left(board):
        print("It's a draw!")
        break

    # Human Turn (X)
    while True:
        try:
            move = int(input("Enter your move (1-9): ")) - 1
            if 0 <= move <= 8 and board[move] == ' ':
                board[move] = 'X'
                break
            else:
                print("Invalid move. The slot is either taken or out
of bounds.")
        except ValueError:
            print("Please enter a valid number between 1 and 9.")

    if not is_moves_left(board) or evaluate(board) == -10:
        continue

    # AI Turn (O)
    print("AI is thinking...")
    ai_move = find_best_move(board)
    if ai_move != -1:
        board[ai_move] = 'O'

if __name__ == '__main__':

```

```

try:
    play_game()
except KeyboardInterrupt:
    print("\nGame exited.")
    sys.exit()

print("Mohammed Shahan")
print("1BM25CS484")

```

OutPut:

```

... Welcome to the Tic-Tac-Toe Simulation!
Positions are numbered 1-9 from top-left to bottom-right.
Initial setup complete. It is X's turn.

Press Enter to make your move...

  O |   | X
  ---|---|---
  X |   |
  ---|---|---
  X | O | O

Enter your move (1-9): 5

  O |   | X
  ---|---|---
  X | X |
  ---|---|---
  X | O | O

Congratulations! You won!
Mohammed Shahan
1BM25CS484

```